

NESNEYE YÖNELİK PROGRAMLAMA

Overloading ve Overriding

Emir Öztürk

ÇOK BİÇİMLİLİK

- Aşırı yükleme (Overloading)
- Çiğneme* (Overriding)

*<http://bilgisayarkavramlari.sadievrenseker.com/2008/11/22/cignemek-overriding/>

Aşırı yükleme (Overloading), aynı isme sahip birden fazla metodun farklı imzalarla tanımlanabilmesi yeteneğidir. Yani aynı sınıf içinde aynı isimli ama farklı parametre listelerine sahip birden fazla metot yazılabilir.

Overloaded metotlar derlenme anında bağlanır (compile-time binding / static dispatch). Derleyici, hangi metodun çağırılacağına derleme sırasında karar verir. Çiğneme (Overriding), bir üst sınıfın metodunun alt sınıf tarafından yeniden yazılıp kullanılabilmesidir. Alt sınıf, üst sınıfın davranışını kendi ihtiyacına göre özelleştirir.

Override edilmiş metotlar çalışma anında bağlanır (runtime binding / dynamic dispatch). Hangi metodun çağırılacağı, nesnenin gerçek türüne göre program çalışırken belirlenir.

METOT İMZASI

- Dönüş türü
- Adı
- Aldığı parametreler

Fonksiyon: `void imzasiOlanFonksiyon(int a, double b, char c){
}`

Prototip: `void imzasiOlanFonksiyon(int, double, char);`

İmza: `imzasiOlanFonksiyon(int, double, char)`

Bir metodun imzası şu üç unsurdan oluşur: dönüş türü, metodun adı ve aldığı parametreler (tür + sayı + sıra).

Parametrelerin türünün veya sayısının değişmesi, metodun imzasını değiştirir.

Bu, aşırı yüklemenin temel koşuludur.

Dönüş türü tek başına imzayı değiştirmez; derleyici dönüş türüne bakarak hangi metodun çağırılacağını belirleyemez.

Örneğin `int topla(int a, int b)` ile `double topla(int a, int b)` aynı imzaya sahip kabul edilir; bu geçerli bir overloading değildir.

AŞIRI YÜKLEME

- İmza farklılıkları
 - Aynı isim
 - Parametre adı farklılıkları
 - Dönüş türü farklılıkları
 - Parametre türü farklılıkları
 - Parametre sayısı farklılıkları

Aşırı yükleme (overloading), farklı imzalara sahip metotların aynı isimle tanımlanabilmesidir. Aynı sınıf içinde aynı isimli metot birden farklı şekilde yazılabilir.

Parametre adı farklılıkları geçerli değildir. Örneğin `topla(int x, int y)` ile `topla(int a, int b)` aynı imzaya sahiptir; bu overloading sayılmaz.

Yalnızca dönüş türü farklılıkları da geçerli değildir. Derleyici, metot çağırıldığında dönüş türüne bakarak hangi versiyonun çalışacağına karar veremez.

Parametre türü farklılıkları geçerlidir. Örneğin biri `int`, diğeri `double` alıyorsa bu farklı imzalıdır.

Parametre sayısı farklılıkları geçerlidir. İki parametre alan ile üç parametre alan aynı isimli metotlar birbirinden farklıdır.

AŞIRI YÜKLEME

```
class Matematik{
    public static int topla(int x,int y){
        return x+y;
    }
    public static int topla(int z,int t){
        return z+t;
    }
}
```

```
class Matematik{
    public static int topla(int x,int y){
        return x+y;
    }
    public static double topla(int x,int y){
        return x+y;
    }
}
```

Overloaded metotların aynı isimli olması şarttır. İsim farklıysa bu zaten başka bir metottur, overloading değildir.

Parametre adlarının farklı olması (örn. x, y vs z, t) imzayı değiştirmez. Her iki metot da iki int aldığı için imzaları özdeştir.

Yalnızca dönüş türü farklı olan iki metodun çağırıldığı noktada derleyici hangi versiyonu kullanacağını bilemez, bu nedenle dönüş türü farklılığı geçersizdir.

AŞIRI YÜKLEME

```
class Matematik{
    public static int topla(int x,int y){
        return x+y;
    }
    public static int topla(double x,int y){
        return (int)x +y;
    }
    public static int topla(int x,double y){
        return x+(int)y;
    }
    public static int topla(int x,int y,int z){
        return x+y+z;
    }
}
```

Parametre türleri farklı olmalıdır ya da parametre sayısı farklı olmalıdır — bu iki koşuldan biri sağlandığında overloading geçerlidir.

Örneğin iki int alan ve üç int alan aynı isimli iki metot tanımlamak geçerlidir çünkü parametre sayıları farklıdır.

Parametrelerin sırası da imzayı etkiler. (double, int) alan metot ile (int, double) alan metot aynı imzaya sahip değildir; bu da geçerli bir overloading örneğidir.

GENERICS

```
class Ekran{
    void EkranaGoster(int a){
        System.out.println(a);
    }
    void EkranaGoster(float a){
        System.out.println(a);
    }
    void EkranaGoster(double a){
        System.out.println(a);
    }
    void EkranaGoster(String a){
        System.out.println(a);
    }
}
```

```
class Ekran<T>{
    void EkranaGoster(T a){
        System.out.println(a);
    }
}
```

```
var liste = new Ekran<Integer>();
var liste = new Ekran<String>();
```

Bir metodun aynı sayıda parametre alıp yalnızca parametre türleri farklıysa, overloading yerine generic'ler tercih edilebilir; böylece her tür için ayrı metot yazmak zorunda kalınmaz.

Generic kullanmanın avantajı: Aynı metot implementasyonunu birden fazla tür için tekrar yazmak gerekmez, kod tekrarı (code duplication) azalır.

Generic kullanmanın dezavantajı: T türü her türden parametre kabul edeceği için tür güvenliği (type safety) azalır; çalışma zamanında hata alma ihtimali artar.

Generic metotlar içinde türe göre dallanma (if (T instanceof ...)) gerekiyorsa, overloading kullanmak daha doğru bir tasarımdır.

Generic'ler, overloading'in doğrudan alternatifi değildir. Arayüz (interface) ve sınıf (class) ile de birlikte kullanılabilirler.

GENERICS

```
class Ekran{
    void EkranGoster(int a){
        System.out.println(a);
    }
    void EkranGoster(float a){
        System.out.println(a);
    }
    void EkranGoster(double a){
        System.out.println(a);
    }
    void EkranGoster(String a){
        System.out.println(a);
    }
}
```

```
class A{
}
public class Main {
    public static void main(String[] args) {
        Ekran t = new Ekran();
        A a = new A();
        t.EkranGoster( = 3);
        t.EkranGoster( = 3.0);
        t.EkranGoster( = a);
    }
}
```

Soldaki örnekte overloading kullanılmış olup her metot yalnızca belirli türdeki parametreleri kabul etmektedir.

Bu yaklaşımda derleme zamanı tür güvenliği sağlanır; yanlış tür gönderilirse derleme hatası alınır.

GENERICS

```
class Ekran<T>{
    void EkranGoster(T a){
        System.out.println(a);
    }
}
```

```
class A{
}
public class Main {
    public static void main(String[] args) {
        Ekran t = new Ekran();
        A a = new A();
        t.EkranGoster( = 3);
        t.EkranGoster( = 3.0);
        t.EkranGoster(a);
    }
}
```

Generic ile yazılmış metotta ise istenilen türde parametre gönderilebilmektedir; bu esneklik sağlar.

Ancak bu esneklik, çalışma zamanında hata oluşturma ihtimalini artırır. Yanlış tür gönderilse bile derleme sırasında hata alınmaz, sorun çalışma anında ortaya çıkar.

AŞIRI YÜKLENEBİLEN METOTLAR

- Eleman fonksiyonlar (Metotlar)
- Yapıcılar
- main metodu
- Yıkıcı
- CPP için operatörler

Eleman fonksiyonlar (sıradan metotlar) aşırı yüklenebilir; bu en yaygın overloading kullanımıdır.

Yapıcı metotlar (constructor'lar) parametre alabildiği için farklı parametre türleri veya sayılarıyla aşırı yüklenebilir.

main metodu teknik olarak aşırı yüklenebilir; ancak JVM yalnızca String[] parametrelili versiyonu çalıştırır, diğerleri normal metot gibi davranır.

Yıkıcı metotlar (destructor) parametre alamadıkları ve dönüş türleri olmadığı için aşırı yüklenemezler.

C++ dilinde operatörler de aşırı yüklenebilir; Java ve C# gibi dillerde bu özellik daha kısıtlıdır.

AŞIRI YÜKLEME

```
class Zaman{
    public Zaman(int baslangic){

    }
    public Zaman(int baslangic,int bitis){

    }
    public Zaman(double baslangic, double bitis){

    }
}
```

Farklı parametre kombinasyonlarıyla birden fazla constructor tanımlanabilir; nesne oluşturulurken hangi verinin sağlandığına göre uygun constructor otomatik çağrılır.

Bu sayede nesne farklı başlangıç durumlarıyla esnek biçimde oluşturulabilir.

OPERATÖR AŞIRI YÜKLEME

- Dil üzerinde önceden tanımlı operatörlerin farklı iş yapması için kullanılır.
- Kod okunurluğu artar.
- Kod anlaşılabilirliğinin azalma ihtimali bulunmaktadır.

Operatör aşırı yükleme, dil üzerinde önceden tanımlı operatörlerin (+, -, *, == vb.) kullanıcı tanımlı türler için farklı davranış sergilemesini sağlar.

Kod okunurluğunu artırır. Örneğin iki vektörü toplamak için toplama(v1, v2) yerine doğrudan v1 + v2 yazılabilir.

Kod anlaşılabilirliğinin azalma ihtimali de bulunmaktadır; aşırı kullanım veya yanlış tanımlar kodun davranışını tahmin edilemez hale getirebilir.

OPERATÖR AŞIRI YÜKLEME

```
class Kisi{
    std::string ad;
    int yas;
public:
    void setYas(int yas){
        this->yas = yas;
    }
};

int main(int argc, const char * argv[]) {
    Kisi k1;
    Kisi k2;
    k1.setYas(30);
    k2.setYas(20);
    std::cout << k1+k2 << std::endl; ❌ Invalid operands to binary expression ('Kisi' and 'Kisi')
    return 0;
}
```

Örnekte + operatörünün özel bir sınıf için nasıl overload edildiği gösterilmektedir. Yapıcı metotlar ve eleman fonksiyonlar gibi operatörler de farklı parametre türleriyle aşırı yüklenebilir.

Operatör overloading yalnızca C++ gibi dillerde doğrudan desteklenir; Java'da operatör overloading yoktur, C#'ta ise kısıtlı biçimde mevcuttur.

OPERATÖR AŞIRI YÜKLEME

```
class Kisi{
    std::string ad;
    int yas;
public:
    void setYas(int yas){
        this->yas = yas;
    }
    int getYas(){
        return this->yas;
    }
    int operator+(Kisi &k){
        return this->yas+k.getYas();
    }
};

int main(int argc, const char * argv[]) {
    Kisi k1;
    Kisi k2;
    k1.setYas(30);
    k2.setYas(20);
    std::cout << k1+k2 << std::endl;
    return 0;
}
```

Operatör aşırı yüklemesinin, normal eleman fonksiyon overloading ile aynı kurallara tabi olduğu görülmektedir.

Operatörü temsil eden özel fonksiyon adı (operator+, operator== vb.) kullanılır; bu fonksiyon da parametre türü veya sayısına göre overload edilebilir.

ÇİĞNEME (OVERRIDING)

- Temel Sınıf
- Kalıtım
- Temel Sınıfın metod özelliklerinin değişimi
- Sınıfların üst sınıflara olan bağlantısı

Overriding, temel sınıfta (parent/super class) bulunan bir metodun, alt sınıfta (child/sub class) yeniden tanımlanmasıyla gerçekleşir.

Amaç: Alt sınıflarda metodu özelleştirmek, üst sınıfın genel davranışını alt sınıfa özgü hale getirmektir.

Eğer alt sınıf ilgili metodu override etmezse, nesne üst sınıfın metodunu kullanmaya devam eder; bu kalıtımın doğal davranışdır.

Override, sınıflar arasındaki kalıtım (inheritance) ilişkisini temel alır; kalıtım olmadan override olmaz.

(ÇİĞNEME) OVERRIDING

- Dönüş türünün aynı olması
- Aldığı parametre değişirse overloading
- Final anahtar kelimesi

Override edilen metodun imzası üst sınıfın methoduyla aynı olmalıdır (aynı isim, aynı parametre listesi, aynı dönüş türü).

Eğer alt sınıfta farklı bir imzayla metod yazılırsa bu yeni bir metottur, overriding değildir.

Aynı sınıf içinde farklı imzalarla yazılan metotlar ise overloading kapsamına girer. final anahtar kelimesiyle işaretlenmiş metotlar override edilemez. Bu, üst sınıfın metodunun alt sınıflar tarafından değiştirilmesini engeller.

(ÇİĞNEME) OVERRIDING

```
class A{
    void metot(){
        System.out.println("A metodu");
    }
}
class B extends A{
    void metot(){
        System.out.println("B metodu");
    }
}
class C extends A{
}
```

```
public class Main {
    public static void main(String[] args) {
        A a = new A();
        a.metot();

        B b = new B();
        b.metot();

        C c = new C();
        c.metot();
    }
}
```

A metodu

B metodu

A metodu

A sınıfından oluşturulan bir nesne üzerinde metot() çağrıldığında, A sınıfının metodunun çalıştığı görülür.

B sınıfından oluşturulan bir nesnede ise B sınıfı metodu override ettiği için B'nin versiyonu çalışır.

C sınıfı metodu override etmemişse, C nesnesi üzerinde metot() çağrıldığında kalıtım zincirinde yukarı çıkılır ve A sınıfının metodu çalışır.

Bu davranış dinamik bağlama (dynamic dispatch / late binding) ile sağlanır; hangi metodun çalışacağı derleme anında değil çalışma anında belirlenir.

(ÇİĞNEME) OVERRIDING

```
class A{
    void metot(){
        System.out.println("A metodu");
    }
}
class B extends A{
    void metot(){
        System.out.println("B metodu");
    }
}
class C extends A{
}
```

```
public class Main {
    public static void fonksiyon(A nesne){
        nesne.metot();
    }
    public static void main(String[] args) {
        A b = new B();
        b.metot();
        fonksiyon(b);
    }
}
```

B metodu

B metodu

Referans türü A olarak belirlenip nesne B sınıfından oluşturulsa bile (A obj = new B()), çalışma anında nesnenin gerçek türü (B) esas alınır.

Bu durum override edilmiş metodun çağırılmasını değiştirmez; nesnenin hafızadaki gerçek tipi belirleyicidir.

(ÇİĞNEME) OVERRIDING

```
class A{
    void metot(){
        System.out.println("A metodu");
    }
}
class B extends A{
    void metot(){
        System.out.println("B metodu");
    }
}
class C extends B{
}
```

```
public class Main {
    public static void fonksiyon(A nesne){
        nesne.metot();
    }
    public static void main(String[] args) {
        A c = new C();
        c.metot();
        fonksiyon(c);
    }
}
```

B metodu

B metodu

Çok seviyeli kalıtımda (A → B → C gibi) metodun en son override edildiği noktadaki versiyon çalıştırılır.

Örneğin C sınıfı, B sınıfından kalıtılmış ve B de metodu override etmişse; C nesnesi üzerinde metot çağırıldığında B'nin versiyonu çalışır (C override etmediği sürece).

C nesnesinin A türünde tanımlanması da sonucu değiştirmez; yine en son override noktası olan B'nin metodu çalışır.

ÇİĞNENEMEYEN METOTLAR

► final anahtar kelimesinin kullanıldığı metotlar

```
class A{
    final void metot(){
        System.out.println("A metodu");
    }
}
class B extends A{
    void metot(){
        System.out.println("B metodu");
    }
}
```

Bir metot final anahtar kelimesiyle tanımlandığında, bu metot hiçbir alt sınıf tarafından override edilemez.

Bu kısıtlama, üst sınıfın kritik bir davranışının alt sınıflarca kasıtsız veya kasıtlı olarak değiştirilmesini önlemek için kullanılır.

final metot hâlâ miras alınabilir ve kullanılabilir; yalnızca yeniden yazılması engellenir.